

Elektronische Schaltungen entwerfen mit SKiDL, KiCad und Claude Code — ein Werkstattbericht

Kurzfassung: Schaltung als Python beschreiben, Netzliste automatisch erzeugen, in KiCad anordnen, routen, fertigen lassen. Vom Schaltungswunsch zur bestellbaren Platine — in einer Sitzung am Schreibtisch, geführt von Claude im Dialog.

Ausgangslage — Platinenlayout war bisher Klickarbeit

Wer eine kleine Schaltung wie einen **LED-Blinker**, ein **Mikrocontroller-Board** oder ein **Sensor-Modul** als richtige Platine wollte, kam an einem EDA-System (Electronic Design Automation) nicht vorbei. Programme wie **KiCad**, **EAGLE**, **Altium** oder **Proteus** sind enorm leistungsfähig — aber auch hier lag genau darin die Hürde:

- ☒ **Schaltplan-Editor und Layout-Editor** als zwei getrennte Welten, dazwischen Bauteilbibliotheken, Footprints, Netzlisten.
- 📐 Man musste sich **Symbole platzieren, Pins verbinden, Bauteilreferenzen vergeben, Netze benennen, Footprints zuordnen** — bevor überhaupt die erste Leiterbahn auf dem Tisch lag.
- ⚡ Selbst „triviale“ Schaltungen — *ein 555-Blinker, ein Spannungsregler mit zwei Elkos, eine LED-Matrix* — bedeuteten **stundenlanges Klicken** durch Schaltplan, Netzlistencheck, Footprintzuordnung, Layout, DRC, Plot.
- 📺 Wer es lernen wollte, kam an YouTube-Tutorials, Volkshochschulkurs oder einem entnervten Bekannten kaum vorbei.

⚠️ Kurz gesagt: **Ein EDA-System musste man weitgehend beherrschen**, bevor die erste eigene Platine den Briefkasten erreichte. Genau das hat viele Hobbyelektroniker, Lehrkräfte und Maker jahrelang abgeschreckt.

Mit der Kombination **SKiDL + KiCad + Claude Code** ändert sich diese Einstiegshürde — darum geht es in diesem Bericht.

Worum geht's

Statt im Schaltplan-Editor zu klicken, wird die Schaltung als **==Python-Skript==** beschrieben. **SKiDL** (Schematics-Kicad-Description-Library) ist eine Python-Bibliothek, die

- **Bauteile aus den KiCad-Standardbibliotheken** instantiiert,
- **Netze** (= elektrische Verbindungen) verwaltet,
- daraus eine **KiCad-Netzliste** erzeugt,
- die dann direkt in den **KiCad-PCB-Editor** importiert wird — ohne den Umweg über den Schaltplan-Editor.

💡 **Kerngedanke:** Die Schaltung entsteht im **Dialog mit Claude** als Code, das Layout im **direkten Klickkontakt mit der Platine** — beides versionierbar, beides nachvollziehbar.

Vorgehen — der 555-Blinker als Beispiel

Klassische Lehrbuchschaltung, didaktisch perfekt: ein **NE555** als astabiler Multivibrator, der eine LED im Sekundenrhythmus blinken lässt.

1. Schaltung als Python

```
from skidl import Part, Net, generate_netlist, TEMPLATE, set_default_tool, KICAD8

set_default_tool(KICAD8)

vcc = Net('VCC'); gnd = Net('GND')
trig = Net('TRIG'); out = Net('OUT')

r_tmpl = Part('Device', 'R', dest=TEMPLATE,

footprint='Resistor_THT:R_Axial_DIN0207_L6.3mm_D2.5mm_P10.16mm_Horizontal')

R1 = r_tmpl(value='10k', ref='R1')
R2 = r_tmpl(value='10k', ref='R2')
R3 = r_tmpl(value='470', ref='R3')
U1 = Part('Timer', 'NE555P', footprint='Package_DIP:DIP-8_W7.62mm')
# ... (Kondensatoren, LED, Stiftleiste analog)

U1['VCC'] += vcc; U1['GND'] += gnd
U1['R'] += vcc          # Reset = VCC: nicht resetten
U1['TR'] += trig
U1['THR'] += trig       # Threshold und Trigger zusammen → astabil
# ... (Timing-Netzwerk, LED-Stufe)

generate_netlist(file_='blinker_555.net')
```

 **Was hier zählt:** Die Schaltung steht als **lesbarer Text**. Frequenz ändern? Eine Zahl. Zweite LED? Drei Zeilen. Versionsverwaltung mit Git? Selbstverständlich.

2. Netzliste erzeugen und importieren

```
py blinker_555.py
```

→ **blinker_555.net** entsteht.

In KiCad: **PCB Editor** → **File** → **Import** → **Netlist...** → Knopf „**Laden und testen**“ (Trockenlauf), dann „**Update PCB**“. Die Bauteile erscheinen am Rand der Platine mit gelben **Luftlinien** dazwischen.

3. Bauteile platzieren

Mit Hotkey **M** (Move) und **R** (Rotate) werden die Bauteile so angeordnet, dass sich die Luftlinien möglichst wenig kreuzen — Faustregel: **kleine Bauteile nah am IC, Kondensatoren direkt an den Pin, Anschlüsse zum Rand**.

4. Leiterbahnen ziehen

Hotkey	Wirkung
X	Routing starten
Esc	Beenden
D	Bahn verschieben
B	Kupferzone füllen

Empfohlene Reihenfolge: **VCC**, dann die kleinen **Signalknoten**, dann der **Ausgang zur LED**. **GND** wird als ==Kupferfläche== auf der Rückseite gelöst — ein Rechteck über die ganze Platine, Net **GND** zugewiesen, mit **B** gefüllt. Alle GND-Pads verbinden sich automatisch.

5. Designregelprüfung (DRC)

Strg + B → DRC läuft. Ziel: **0 Fehler, 0 ungenutzte Verbindungen**. Solange Fehler stehen, wird nicht plottet.

6. Fertigungsdaten

File → **Plot...** → Gerber-Lagen exportieren. Zusätzlich **File** → **Fabrication Outputs** → **Drill Files** für die Bohrdaten. Alles in einen Ordner, daraus ein ZIP — und beim Fertiger hochladen.

Wie sich das im Arbeiten anfühlt

Im Test eine simple Aufgabe an Claude:

„Ich möchte einen 555-Blinker mit etwa 5 Hz und einer LED am Ausgang.“

Claude:

- legt das **SKiDL-Skript** mit den passenden Bauteilwerten an,
- erklärt die **Frequenzformel** $f = 1,44 / ((R1 + 2 \cdot R2) \cdot C1)$ mit eingesetzten Zahlen,
- richtet die **KiCad-Umgebungsvariablen** für die Bibliothekssuche ein,
- führt das Skript aus, sieht den Fehler **Can't open file: Device**,
- erkennt: **SKiDL-Default ist KiCad 5**, muss auf KiCad 8 umgestellt werden,
- erkennt: **NE555P-Pins heißen R und CV**, nicht **RESET** und **CTRL**,
- korrigiert das Skript, Lauf ohne Fehler.

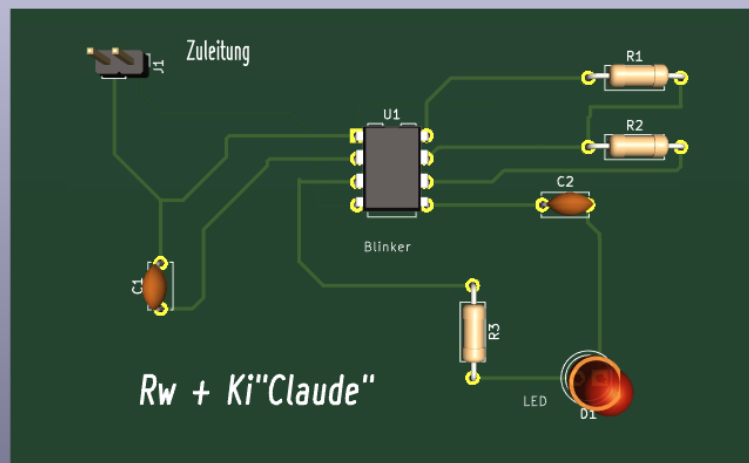
Anschließend Folgewünsche im Plauderton:

„Wie zippe ich die Gerber-Dateien?“

→ Claude schaut in den Projektordner, **sieht dass die Bohrdateien fehlen**, weist darauf hin, packt das vorläufige ZIP, erklärt wie die **.dr1** zu erzeugen sind, und aktualisiert das ZIP nach der Rückmeldung.

🔑 **Was an dieser Schleife auffällt:** Claude **liest tatsächlich den Projektordner**, statt aus dem Bauch zu raten. Wenn eine Datei fehlt oder ein Bibliothekspfad nicht stimmt, fließt das in die nächste Antwort ein. Das ist näher an einem **Werkstattsgesellen** als an einem Skript-Generator.

Das Ergebnis



Die fertige Platine in der **KiCad-3D-Ansicht**: NE555 im DIP-8-Gehäuse mittig, Timing-Widerstände R1/R2 und Entstörkondensator C2 rechts, Elko C1 links, LED mit Vorwiderstand R3 rechts unten, Versorgung über Stiftleiste „Zuleitung“ oben links. Größe ca. **70 × 40 mm** — bewusst nicht minimiert, sondern lesbar für den Unterricht.

☒ **Vom „ich möchte einen Blinker“ zur bestellbaren Gerber-ZIP-Datei in einer einzigen Sitzung.**

Wo das stark wird — Projektideen für den Unterricht

Anwendungsfeld	Beispielnutzen
Erste eigene Platine in einer Schulstunde	555-Blinker, Spannungsteiler, Audioverstärker mit LM386 — alles in einer Doppelstunde vom Code bis zum Gerber.
Schaltung als Code im Informatik-Unterricht	Brückenschlag E-Technik ↔ Programmierung — der gleiche Code, lesbar, kommentierbar, versionierbar.

Anwendungsfeld	Beispielnutzen
Designvarianten	„Mach das Gleiche mit zwei wechselblinkenden LEDs" — Schüler ändern Code, vergleichen Layouts.
Reparieren statt wegwerfen	Defekte Steuerplatine nachbauen — Schaltbild abfotografieren, Claude überträgt in SKiDL, neue Platine fertigen lassen.
Mikrocontroller-Boards	ATtiny, ESP32, Arduino-kompatibel — minimal-PCB für eigene Schülerprojekte.
Sensorik und IoT	Temperatur-, Licht-, Bewegungsmelder mit eigenem Layout — kein Stecker-Spaghetti mehr.

Aussichten

Das Schöne an SKiDL + Claude: die Schaltung ist **==Quelltext==**, und damit gilt:

Was Code ist, ist **versionierbar, vergleichbar, generierbar, diff-bar**.

Damit verschiebt sich die Arbeitsteilung:

- **Der Lernende** denkt in *Funktion, Spannungen, Frequenzen, Bauteilwerten*.
- **Der Assistent** übernimmt die Übersetzung in *Symbol, Pin, Net, Footprint*.

⚠ Das **ersetzt Elektronik-Verständnis nicht** — jemand muss die Schaltung weiterhin auf Plausibilität prüfen, Datenblätter konsultieren, Stromgrenzen einhalten — aber es **nimmt die mühsame Klick-Arbeit ab**.

Mittelfristig spannend

- 📦 **Bibliothek wiederverwendbarer Schaltungsblöcke** als SKiDL-Module (Stromversorgung, Pegelwandler, OPV-Stufe...)
- 🖱 **Simulation mit ngspice direkt aus SKiDL** — Schaltung rechnerisch prüfen, bevor sie geätzt wird
- 🔗 **Verkettung mit dem Fusion-Workflow**: Platenumriss in Fusion mechanisch konstruieren, in KiCad als Edge.Cuts importieren — Gehäuse und Platine aus einem Guss

Fazit

🔗 Was vor zwei Jahren noch nach „Profi-Werkzeug für Spezialisten" klang — **„beschreib mir die Schaltung, ich layout sie dir"** — funktioniert heute mit einem freien Tool-Stack und einer Mittagspause Setup.

Und es bringt Schülern und Lehrenden gleichermaßen **richtig Freude an der Elektronik** zurück.



Erstellt: 2026-06-12 · SKiDL 2.2.3 + KiCad 8.0.8 + Claude Code + Rw